Project Chronos

When Clocks Lie and Messages Wander

The Story

Five friends ($N = 5$) sit in five different networks. In exactly two minutes, the university's course registration portal will open. The portal is unforgiving: seats vanish in milliseconds, and if requests arrive with inconsistent timestamps, the server bans your IP.

There are two problems. First, every laptop clock shows a slightly different time. Some are off by a few milliseconds; one might be off by several seconds. Nobody knows who is right. Second, the friends must agree on who will bid for which elective by exchanging messages. Because the network is erratic, messages do not always arrive in the order they were sent. If a friend acts on a message before receiving the hidden history that preceded it, the coordination falls apart.

Before the portal opens, they must mathematically align their clocks and process their messages in an order that strictly respects cause and effect.

The Task

Simulates all five nodes, read a `seed.json` file containing initial clock values, a tolerance threshold θ, and a scrambled queue of incoming messages for each node. You must use delays (e.g., pausing for fractions of a second) to stream the events to the terminal in real-time. The user should watch as nodes poll each other, messages traverse the network, and causal buffers hold and release data.

Step 1: Node 0 acts as the coordinator. It calculates a trusted time average and issues corrections to bring the clocks together.

Step 2: Each node processes its queue of incoming messages. If a message arrives too early (missing its causal history), the node must hold it in a buffer. As missing messages arrive, the buffer must be recursively checked and emptied.

Step 3: After processing all messages, each node calculates a final numerical payload to send to the portal.

The Rules

1. The Clock Alignment

Let T_i be Node i's initial clock. Node 0 keeps only the nodes whose clocks are within θ of its own:

$$S = \{\, i \mid |T_i - T_0| \leq \theta \,\}$$

Node 0 computes a target time from this trusted set (using integer floor division):

$$T_{\text{target}} = \left\lfloor \frac{1}{|S|} \sum_{i \in S} T_i \right\rfloor$$

Each trusted node receives a correction offset $\delta_i = T_{\text{target}} - T_i$. Nodes outside the trusted set are ignored and receive an offset of 0. A node's aligned clock is $T_i^{\text{sync}} = T_i + \delta_i$.

2. The Message Ordering

Each node i keeps an array V_i of N counters, starting at zero. When Node i receives a message from Node j carrying the sender's array V_{msg}, it checks two conditions:

$$V_{\text{msg}}[j] = V_i[j] + 1 \quad \text{and} \quad \forall\, k \neq j,\ V_{\text{msg}}[k] \leq V_i[k]$$

If both hold, the message is processed, and Node i updates its array: $V_i[k] = \max(V_i[k], V_{\text{msg}}[k])$ for all k. It then immediately checks if any previously held messages can now be processed. If either condition fails, the message is placed in a buffer.

3. The Final Payload

Once a node has emptied its queue and buffer, it computes:

$$\mathcal{P}_i = \left(T_i^{\text{sync}} \times \textstyle\sum_{k=0}^{N-1} V_i[k] \right) \bmod 9973$$

"You cannot step into the same river twice,
for other waters are continually flowing on."

— Heraclitus